You said:

Hi i am doing my CS409 MP3
I will share the zip file ..
can you pleaseguide me how to go about and explain me in each step.
I want to do it on my own.. just give me instructions and explain

ChatGPT said:

Hi! 😊 Absolutely! We'll work through your **CS 409 MP3 step by step**, and the goal is for you to understand everything and implement it yourself.

I won't simply give you the complete solution or write all the code for you. Instead, I'll guide you like a TA: explain the concepts, help you understand the existing code, give you small tasks, and help you debug when you get stuck.

How we'll approach MP3

Step 1: Understand the assignment
Upload the ZIP file. We'll examine the handout, requirements, starter code, and grading rubric to understand exactly what you need to implement.

Step 2: Understand the codebase
We'll identify the important files, understand how they interact, and determine which functions you need to modify or implement.

Step 3: Implement one piece at a time
For each task, I'll explain the underlying concept, outline the logic in plain English or pseudocode, and let you write the implementation yourself.

Step 4: Test and debug
You'll run your code, share any errors or test results, and we'll work through them together. We'll also discuss edge cases.

Step 5: Verify the requirements
We'll compare your implementation against the assignment requirements and available tests, then review anything that might be missing.

What I need from you first

Upload your CS 409 MP3 ZIP file here.

Please upload the original starter ZIP if possible, including the assignment handout and any README or test files provided by the course.

Once you upload it, I'll start by:

1. Explaining what the assignment is asking you to build.
2. Breaking the requirements into manageable tasks.
3. Explaining the project structure and what each important file does.
4. Identifying the first small task for you to complete.

We'll proceed at your pace. I'll explain unfamiliar concepts from the fundamentals, and **you'll write the code, run it, and understand why it works**.

Don't worry if the code initially looks confusing. We'll take it one step at a time. Upload the ZIP, and let's get started! 🚀

You said:

mp3-main(1).zip
File

ChatGPT said:

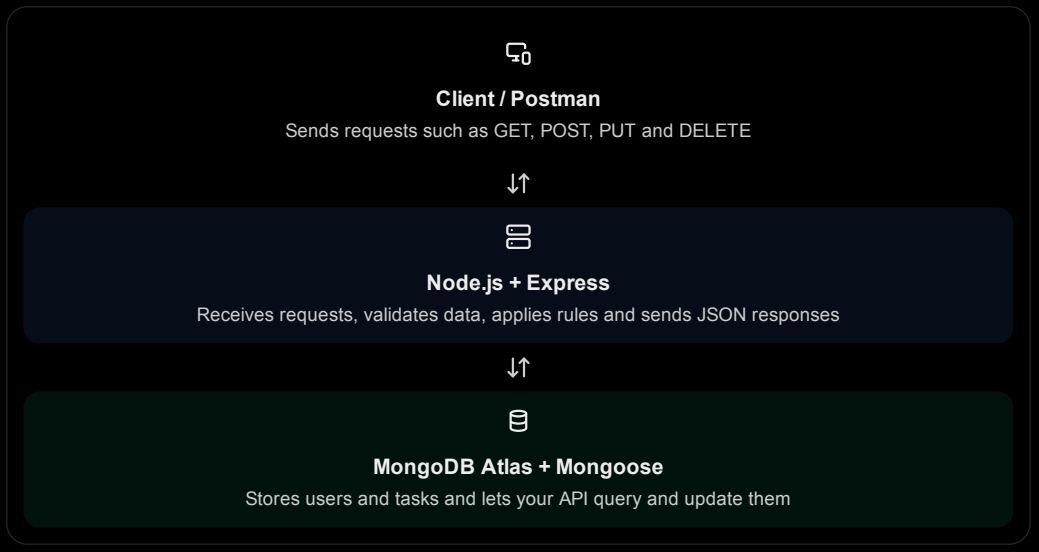
I've received your ZIP file! Let's work through your **CS 409 MP3: APIed Piper** together.

The assignment is to build a task-management REST API using **Node.js, Express, MongoDB Atlas, and Mongoose**. The deadline listed in your handout is **Tuesday, November 3, 2026, at 11:59 PM Central Time**.

I also noticed an important course rule in the handout: this is an individual assignment, and if you use an LLM to help generate code, you must submit your chat logs and answer the related survey questions. We'll focus on explaining concepts, helping you understand the starter code, and guiding you through your own implementation.

Step 1: Understand what you're building

Think of your API as the backend of a to-do application. A client sends HTTP requests, your server processes them, and MongoDB stores the data.



Here are the main concepts we'll use:

- **Node.js:** Runs JavaScript on your server.
- **Express:** Defines API routes and handles HTTP requests.
- **MongoDB Atlas:** Hosts your database in the cloud.
- **Mongoose:** Defines your data models and communicates with MongoDB.
- **REST API:** Provides standard operations for creating, reading, updating, and deleting resources.

Step 2: Understand the data models

Your assignment has two main resources.

User

- `name`
- `email`
- `pendingTasks`
- `dateCreated`

Task

- `name`
- `description`
- `deadline`
- `completed`
- `assignedUser`
- `assignedUserName`
- `dateCreated`

The most challenging part is keeping users and tasks consistent.

For example, suppose a user named Alex has two pending tasks. The user's `pendingTasks` list must contain both task IDs, and each task must point back to Alex using `assignedUser` and `assignedUserName`.

If a task is completed, it stays assigned to Alex but must be removed from the pending list. If Alex is deleted, all of Alex's tasks must become unassigned.

We'll work through these rules carefully rather than trying to implement everything at once.

Step 3: Explore the starter ZIP

I inspected your ZIP. It already contains a substantial amount of starter code, including user and task models, route handlers, shared query helpers, and error handling. **Don't rewrite everything or assume every file is incomplete.** We'll first understand what's there and then identify what needs attention.

Here is your project structure:

- **mp3-main/**
  - **server.js**
    Starts Express, loads environment variables, connects to MongoDB, and mounts the routes.
  - **models/**
    Defines the User and Task database schemas.
    `user.js` · `task.js`
  - **routes/**
    Handles API requests, validation, and user/task operations.
    `users.js` · `tasks.js` · `helpers.js` · `index.js` · `home.js`
  - **README.md and FAQ.md**
    Assignment specifications, setup instructions, edge cases, and grading information.
  - **submit.js**
    Runs the provided self-check against your deployed API.

Step 4: Understand the API operations

These are the endpoints you need to understand and test.

Endpoint	What it does
`GET /api/users`	List users, optionally filtered or sorted
`POST /api/users`	Create a user
`GET /api/users/:id`	Fetch one user
`PUT /api/users/:id`	Replace a user and synchronize task assignments
`DELETE /api/users/:id`	Delete a user and unassign their tasks
`GET /api/tasks`	List tasks with query options
`POST /api/tasks`	Create a task
`GET /api/tasks/:id`	Fetch one task
`PUT /api/tasks/:id`	Replace a task and synchronize its assignment
`DELETE /api/tasks/:id`	Delete a task and update its user's pending list

For example, `POST /api/users` creates a user, while `GET /api/users` retrieves the list. The `:id` in a URL represents a specific resource ID.

The assignment also requires correct HTTP status codes, JSON responses, validation, and query parameters such as `where`, `sort`, `select`, `skip`, `limit`, and `count`.

Step 5: Your first task — prepare your local environment

Let's start with setup before changing any code.

Setup checklist

Check each item as you complete it.

- ◯ Install Node.js 24 LTS and verify with `node -v`.
- ◯ Open a terminal in the extracted project directory and run `npm install`.
- ◯ Create your own GitHub repository using the course's template, then clone it locally.
- ◯ Create or configure your MongoDB Atlas database and connection string.

0 of 4 completed

A few important details:

- Keep the provided `.gitignore` file.
- Create your `.env` from `.env.example` and add your MongoDB connection string.
- Never commit or upload `.env` to GitHub.
- Use the course-supported Node.js version specified in the handout.
- Don't deploy yet. First, get familiar with the code and verify your local setup.

Step 6: Let's inspect the code together

I suggest we begin with `models/user.js` and `models/task.js`. Understanding these two files will make the route handlers much easier to follow.

For instance, in the User schema, you will see:

- `name` and `email` are required.
- `pendingTasks` is a list of task IDs.
- `dateCreated` is assigned automatically and is immutable.

The Task schema has similar defaults, plus its deadline, completion status, and assignment fields.

Your turn: Have you already extracted the ZIP and opened the project in VS Code, or do you need help setting that up first?

You said:

done

ChatGPT said:

Great! 😊 Let's move to the next step. We'll go slowly so you understand the code rather than just following commands.

Step 7: Open and understand the project

Open your project in VS Code. In the terminal, run:

Bash

```
node -v
npm -v
```

These commands tell us whether Node.js and npm are installed and available.

Next, navigate to the project folder and run:

Bash

```
npm install
```

This installs the dependencies listed in `package.json`.

Step 8: Let's start with the database models

Open these two files in VS Code:

- `models/user.js`
- `models/task.js`

Before we start implementing anything, I want you to understand what these files do.

What is a Mongoose schema?

A schema defines the structure of a document stored in MongoDB. For example, a Task might have a name, description, deadline, and completion status.

A schema can also specify:

- Required fields
- Default values
- Data types
- Validation rules
- Whether a field can be modified after creation

The model then lets your application create, retrieve, update, and delete database documents.

Your first mini-task

Open `models/user.js` and `models/task.js`. Don't change anything yet.

Tell me these two things:

1. Did `npm install` finish successfully, or did you get an error?
2. In `models/user.js`, can you find the `pendingTasks` field? What data type is it declared as?

You can paste the relevant schema here if you're unsure. We'll use your answers to move to the next step.

You said:

next

Worked for 7s >

ChatGPT said:

Perfect! Let's continue with **Step 9: Understand how the API handles requests**.

We'll move from the database models to the route handlers. Remember, your goal is to implement the assignment yourself, so we'll understand the flow first and then tackle one operation at a time.

Step 9: Understand the request flow

When a client sends a request to your API, it typically follows this sequence:

1. HTTP request
Example: `POST /api/users` with JSON data

↓

2. Express route
Finds the handler for the URL and HTTP method

↓

3. Validation and database operation
Checks input and reads or modifies MongoDB

↓

4. HTTP response

Returns a status code and usually a JSON body

For example, `POST /api/users` could receive:

JSON

```
{
  "name": "Alex",
  "email": "alex@example.com"
}
```

The handler validates the input, creates the user document, saves it, and returns the appropriate response according to your assignment specification.

Step 10: Explore `routes/users.js`

In VS Code, open:

`routes/users.js`

At the top, you'll see these imports:

JavaScript

```
var User = require('../models/user');
var Task = require('../models/task');
var h = require('./helpers');
var ApiError = h.ApiError;
```

Let's understand them one by one.

Import	Purpose
`User`	Interacts with the users collection in MongoDB.
`Task`	Interacts with the tasks collection.
`h`	Provides shared helper functions for queries, validation, and responses.
`ApiError`	Represents an API error with an HTTP status code and a readable message.

Notice that `users.js` imports both `User` and `Task`. Why? Because changing a user's tasks may require updating the corresponding task documents, too.

Step 10.1: Understand a GET request

Find this section in the file:

JavaScript

```
usersRoute.get(async function (req, res) {
    var result = await h.runListQuery(User, req.query);
    h.send(res, 200, 'OK', result);
});
```

Let's break it down.

- `usersRoute.get(...)` registers a handler for `GET /api/users`.
- `req` contains information about the incoming request.
- `res` is used to send the response.
- `req.query` contains query parameters from the URL.
- `await` waits for the database operation to finish.
- `h.send(...)` formats and sends the response.

For example, a request to `GET /api/users?limit=5` asks for at most five users.

The route delegates the query processing to `h.runListQuery()`, rather than implementing filtering and sorting itself. This is important because the assignment requires those operations to use Mongoose.

Step 10.2: Follow the helper function

Open:

`routes/helpers.js`

Find the function named `runListQuery`.

Your task is to identify where it handles each of these parameters:

- `where`
- `sort`
- `select`
- `skip`
- `limit`
- `count`

Don't modify anything yet. We first want to understand how the existing implementation works.

Your mini-task

Question 1: What does `await h.runListQuery(User, req.query)` do?

- ○ It creates a new user in MongoDB.
- ○ It retrieves users according to the supplied query parameters.
- ○ It deletes users matching the query.

Question 2: Did you find `runListQuery` inside `routes/helpers.js`?

- ○ Yes, I found it.
- ○ No, I need help finding it.

Check answers and continue

Once you've done this, we'll move on to understanding how query parameters are parsed and how Mongoose executes the query.

You said:

next

ChatGPT said:

Let's continue! We'll now focus on **Step 11: Query parameters and how MongoDB retrieves data**.

This is an important part of your CS 409 MP3 because an API should not only return data; it must also support filtering, sorting, selecting fields, and pagination.

Step 11.1: Understand query parameters

Suppose your API has these three users:

Name	Email
Alex	alex@example.com
Priya	priya@example.com
Sam	sam@example.com

A normal request:

`GET /api/users`

returns the list of users.

But query parameters let a client request a specific subset of that data.

Parameter	Purpose	Example
`where`	Filter documents	Find users matching a condition
`sort`	Sort results	Sort by name
`select`	Choose returned fields	Return only name and email
`skip`	Skip initial results	Skip the first two
`limit`	Limit result count	Return at most five
`count`	Request the count of matching documents	Count users matching a filter

Think of these as instructions that modify a database query.

Step 11.2: Understand how a query becomes a database operation

Open `routes/helpers.js` and locate these three functions:

1. `parseJsonParam()`
2. `parseListQuery()`
3. `runListQuery()`

They each have a different responsibility.

`parseJsonParam()`
Converts a JSON query-string value into a JavaScript object and reports malformed JSON as a 400 error.

`parseListQuery()`
Collects the filtering, sorting, field selection, pagination, and count options.

`runListQuery()`
Uses Mongoose methods such as `find()`, `sort()`, `select()`, `skip()`, `limit()`, and `countDocuments()` to execute the query.

Let's trace one request

Suppose the client requests:

Plain text

```
GET /api/tasks?where={"completed":true}&limit=5
```

Conceptually, the server does the following:

1. Parses `where` into an object equivalent to `{ completed: true }`.
2. Parses `limit` into the number `5`.
3. Builds a Mongoose query that filters for completed tasks.
4. Applies a maximum result count of five.
5. Executes the query and returns the results in the response's `data` field.

Notice that the filtering and limiting happen in MongoDB through Mongoose, rather than retrieving every task and filtering the results in JavaScript. This matches an explicit assignment requirement.

One detail to watch: `GET /api/tasks` has a default limit of 100, whereas `GET /api/users` has no default limit. That's why the task route passes `100` to the shared helper.

Step 12: Your turn — practice tracing the query

Question 1

A client sends:

```
Plain text
GET /api/tasks?skip=10&limit=5
```

Which Mongoose methods are responsible for pagination?

- `find()` and `save()`
- `skip()` and `limit()`
- `create()` and `deleteOne()`

Question 2

Why does the helper use `JSON.parse()` for the `where` parameter?

- Because URL query parameters arrive as strings and `where` must be interpreted as a JSON object.
- Because MongoDB only accepts strings for queries.
- Because `JSON.parse()` connects to MongoDB Atlas.

Check my answers

After this, we'll trace `POST /api/users` and understand validation, duplicate emails, and how a user's `pendingTasks` list stays consistent with task assignments. We'll keep the focus on understanding the provided code before you make changes.

You said:

next

Worked for 7s >

ChatGPT said:

Let's move on to **Step 13: Creating a user — `POST /api/users`**.

This is where you'll start connecting the concepts you've learned: Express routes, request bodies, Mongoose models, validation, and database operations.

Step 13.1: Understand the request

Imagine a client wants to register a new user in your task-management API.

It sends this HTTP request:

```
http
POST /api/users
Content-Type: application/json
```

With this JSON body:

```
JSON
{
  "name": "Alex",
```

```
  "email": "alex@example.com"
}
```

The server needs to:

1. Read the request body.
2. Check whether the required fields are present and valid.
3. Check whether the email already exists, if required by the specification.
4. Create the user document.
5. Save it to MongoDB.
6. Return the appropriate HTTP status and response body.

The important distinction is that **receiving a request is not the same as successfully creating a database record**. Validation or database operations can fail, and your API must handle those cases correctly.

Step 13.2: Trace the actual implementation in your starter code

Open `routes/users.js` and find:

JavaScript

```
usersRoute.post(async function (req, res) {
```

Read the statements inside this handler in order. Your starter code already contains an implementation, so **don't replace it or rewrite it yet**.

Here is what each operation in the handler is responsible for:

Operation	Purpose
`validateUserBody(req.body)`	Checks required fields and validates the request data.
`ensureEmailAvailable(fields.email)`	Checks whether the email is already used.
`checkPendingTasks(fields.pendingTasks)`	Verifies that the specified tasks exist and are not completed.
`User.create(fields)`	Creates the user document in MongoDB.
`assignTasksToUser(...)`	Updates task assignments and other users' pending lists as needed.
`h.send(res, 201, ...)`	Sends the successful creation response.

Why do we need all these checks?

Consider three situations:

- **Missing name:** The server should reject the request with a `400 Bad Request`.
- **Duplicate email:** The server should reject the request instead of creating a second user with that email.
- **Completed task in `pendingTasks`:** The server should reject the request because a completed task cannot be pending.

Notice that validation happens before `User.create()`. This prevents invalid requests from creating a user document.

Step 14: Your first code-reading exercise

Let's check whether you understand the flow before moving on to task creation.

1. Why does the handler call `ensureEmailAvailable()` before `User.create()`?

- ○ To prevent duplicate email addresses.
- ○ To sort users by email.
- ○ To assign tasks automatically.

---

2. What does HTTP status code `201` mean?

- The resource was not found.
- A resource was successfully created.
- The request was invalid.

3. Why does `assignTasksToUser()` update both User and Task documents?

- To keep task assignments and users' pending-task lists consistent.
- To create a second copy of every task.
- To delete completed tasks.

Check answers and continue

Next, we'll study `routes/tasks.js`, where you'll learn how task creation, deadlines, completion status, and assignment to users work. We'll continue using your starter code as the reference and identify what the assignment actually expects you to implement or test.

You said:

next

ChatGPT said:

Step 15: Creating a task — `POST /api/tasks`

Now let's study the other main resource in your CS409 MP3: **tasks**.

A task is different from a user because it has additional fields, including a deadline, completion status, and an optional user assignment.

Step 15.1: Understand the request

Suppose a client wants to create this task:

http

```
POST /api/tasks
Content-Type: application/json
```

JSON

```
{
  "name": "Finish MP3",
  "description": "Implement and test the API",
  "deadline": "2026-11-02T23:59:00.000Z",
  "completed": false
}
```

The server must validate the input, create the task, and return the created resource.

Your `models/task.js` defines the task's schema. Think of the schema as the rules describing what a valid task document can contain.

Step 15.2: Understand the task validation

In `routes/tasks.js`, find the function:

JavaScript

```
validateTaskBody(req.body)
```

This function checks the incoming data before the database creates a task.

Your `models/task.js` defines these fields:

Field	Type	Purpose
`name`	String	Task name; required
`description`	String	Description; defaults to `""`
`deadline`	Date	Due date; required
`completed`	Boolean	Whether the task is finished; defaults to `false`
`assignedUser`	String	ID of the assigned user; defaults to `""`
`assignedUserName`	String	Assigned user's name; defaults to `"unassigned"`
`dateCreated`	Date	Automatically set by the server

What should happen with invalid input?

Try reasoning through these cases:

- **Case A:** The request has a name but no deadline.
- **Case B:** The deadline is `"not-a-date"`.
- **Case C:** The request has a valid deadline but `completed` is `"yes"` instead of a Boolean.
- **Case D:** The request specifies an `assignedUser` ID that doesn't belong to an existing user.

The validation and assignment checks should reject invalid values or missing references, rather than silently creating an incorrect task.

Step 15.3: Understand the assignment logic

Look at the following operations in your existing `POST /tasks` handler:

JavaScript

```
var fields = validateTaskBody(req.body);
await resolveAssignment(fields);

var task = await Task.create(fields);
await syncUserPendingTasks(task);
```

Here's the flow:

1. Validate request
Check field types and required values.

↓

2. Resolve assignment
Check the user exists and obtain their name.

↓

3. Create task
Save the document in MongoDB.

↓

4. Synchronize references
Update the assigned user's pending-task list if appropriate.

The key idea is that **a task's assignment and a user's pending-task list must agree**. If an incomplete task is assigned to a user, its ID should appear in that user's `pendingTasks`. A completed task should not be listed as pending.

Step 16: Your turn — inspect and test

Before moving to updates and deletions, do this small exercise in your editor:

1. Open `models/task.js` and identify which fields have defaults.
2. Open `routes/tasks.js` and read `validateTaskBody`, `resolveAssignment`, and `syncUserPendingTasks`.
3. Explain to yourself why the route calls these functions in this order.
4. If your MongoDB connection is configured, test creating one task with a valid request and another with a missing deadline.

One important note: the ZIP you shared already contains substantial route implementations. We should **verify what works and identify any missing requirements before changing code**.

Next, we'll cover `PUT` and `DELETE` requests, including how to update a task without leaving stale references in a user's pending-task list.

You said:

next

ChatGPT said:

Step 17: Updating and deleting tasks

Now we'll look at `PUT` and `DELETE` in `routes/tasks.js`.

These operations are important because your API must keep the database consistent—not just return the right response.

Step 17.1: `PUT /api/tasks/:id`

`PUT` updates an existing task. For example, a client might change the task's name, deadline, completion status, or assigned user.

The route should follow this general sequence:

1. Validate the task ID in the URL and confirm the task exists.
2. Validate the request body.
3. Check that any assigned user exists.
4. Apply the updated fields.
5. Synchronize the affected users' `pendingTasks` lists.
6. Return the updated task.

Why check the ID first? Your assignment's README specifies error precedence: an invalid or nonexistent resource ID should produce `404` before body validation errors are reported.

Step 17.2: Why does updating a task require synchronization?

Suppose user Alex has task `T1` in their `pendingTasks` list.

Now consider three possible updates:

Scenario A: Mark the task completed

The task's `completed` field becomes `true`. Its ID should be removed from Alex's `pendingTasks`.

Scenario B: Reassign the task to Sam

Remove the task ID from Alex's list, update its assignment to Sam, and add the ID to Sam's list if the task is incomplete.

Scenario C: Mark the task incomplete again

If it is still assigned to Alex, add its ID back to Alex's `pendingTasks`.

Your starter code uses `syncUserPendingTasks(task)` to handle this synchronization. Read its implementation carefully and identify how it handles the old assignment versus the new assignment.

Step 18: `DELETE /api/tasks/:id`

Deleting a task means more than removing a document from the `Task` collection.

The route must also remove the deleted task's ID from any users' `pendingTasks` arrays.

For example, before deletion:

```json
{
  "name": "Alex",
  "pendingTasks": ["task123", "task456"]
}
```

If `task123` is deleted, Alex's pending list should no longer contain that ID.

The expected general flow is:

1. Validate the task ID and find the task.
2. Remove its ID from affected users' pending lists.
3. Delete the task document.
4. Return `204 No Content`, without a response body.

Important HTTP status codes

Status	Meaning
`200 OK`	An existing task was retrieved or updated successfully.
`201 Created`	A task was created successfully.
`204 No Content`	A task was deleted successfully; no response body.
`400 Bad Request`	The request contains invalid data.
`404 Not Found`	The requested task or referenced resource does not exist.

Step 19: Check your understanding

Question: Alex has an incomplete task assigned to them. You update the task so that it is assigned to Sam instead. What must happen to keep the database consistent?

- ○ Only change the task's `assignedUser` field.
- ○ Remove the task ID from Alex's pending list, update the assignment, and add it to Sam's pending list.
- ○ Delete and recreate both users.
- ○ Nothing; MongoDB automatically updates every reference.

Check answer and continue

After this, we'll examine query parameters and API testing together—including how to test success cases, invalid inputs, missing IDs, filtering, sorting, and pagination. Testing is how you'll establish whether the implementation meets the assignment requirements rather than just assuming it does.

You said:

next

ChatGPT said:

Step 20: Test your API systematically

You've now learned the main operations for users and tasks. The next step is to verify that your implementation behaves correctly.

For your CS409 MP3, don't test only whether a request succeeds. Also test whether invalid requests return the correct status code and leave the database unchanged.

Step 20.1: Prepare your testing environment

Use whichever API client you prefer, such as Postman, VS Code's REST Client, or `curl`.

Before testing, confirm:

- Your Node.js server starts without errors.
- Your MongoDB connection succeeds.
- You know the API's base URL and route prefix from `server.js` and `routes/index.js`.
- You can inspect the database or API responses to verify the results.

Don't assume the route prefix is `/api` without checking your starter code.

Step 20.2: Test the main endpoints

Your `routes/index.js` confirms that your API uses the `/api` prefix, so the base URL is `http://localhost:3000/api`.

Start with these tests. Replace example IDs with real IDs returned by your API.

Test 1 — Health check

```http
GET http://localhost:3000/api/
```

Inspect `routes/home.js` to confirm the exact health-check path and expected response.

Test 2 — List users

```http
GET http://localhost:3000/api/users
```

Expected: `200 OK`, with a JSON response containing `message` and `data`.

Test 3 — List tasks

```http
GET http://localhost:3000/api/tasks
```

Expected: `200 OK`. The task list should be empty if you have no tasks yet.

Test 4 — Filter tasks

```http
GET http://localhost:3000/api/tasks?where={"completed":true}
```

Expected: `200 OK`, containing only completed tasks. If your client has trouble with the URL, encode the JSON query parameter.

Test 5 — Invalid task ID

```http
```

```
GET http://localhost:3000/api/tasks/not-an-id
```

Expected: `404 Not Found`, according to the assignment's ID-handling requirements.

Step 20.3: Check error handling

Try these additional cases:

Request	Expected result
Create a user without `name`	`400 Bad Request`
Create another user with an existing email	`400 Bad Request`
Create a task without `deadline`	`400 Bad Request`
Fetch a nonexistent task ID	`404 Not Found`
Delete an existing task successfully	`204 No Content`

For every failed POST or PUT, verify that **the database was not partially modified**. This matters especially when a request changes references between users and tasks.

Step 21: Test query parameters

Your README requires collection endpoints to support combinations of these parameters:

Parameter	Example	Purpose
`where`	`{"completed":true}`	Filter documents
`sort`	`{"name":1}`	Sort by name ascending
`select`	`{"_id":0}`	Exclude the `_id` field
`skip`	`10`	Skip the first 10 results
`limit`	`5`	Return at most 5 results
`count`	`true`	Return the number of matching documents

Try combining `sort`, `skip`, and `limit`. The assignment expects Mongoose to perform these operations in the database query, not JavaScript to retrieve everything and filter it afterward.

One detail to remember: the default limit is **100 tasks but unlimited users**.

Your next step

Before we proceed to the final testing and submission checks, tell me where you currently stand.

- (•) I haven't run the server or tested endpoints yet.
- ( ) The server runs, and basic GET requests work.
- ( ) The server or requests are producing errors.
- ( ) I've tested most endpoints and want to check assignment requirements.

Optional: share the error message or what you have tested.

For example: MongoDB connection error, GET /api/tasks returns 200...

Continue with my current progress

You said:

next

ChatGPT said:

Step 22: Verify the assignment requirements before submitting

Let's move from learning individual endpoints to checking whether your CS409 MP3 implementation satisfies the specification.

Since you want to complete the assignment yourself, we'll use a **requirements checklist** rather than replacing your code with a finished solution.

Step 22.1: Check the API behavior

For each item below, inspect your implementation and run a test if possible.

MP3 self-check 0 of 8 checked

- ◯ **Users CRUD**
  Create, retrieve, update, and delete users.
- ◯ **Tasks CRUD**
  Create, retrieve, update, and delete tasks.
- ◯ **Validation**
  Reject missing required fields, incorrect types, and invalid values.
- ◯ **HTTP status codes**
  Return the appropriate 200, 201, 204, 400, and 404 responses.
- ◯ **Query parameters**
  Test where, sort, select, skip, limit, and count.
- ◯ **Reference consistency**
  Keep assignedUser, assignedUserName, and pendingTasks synchronized.
- ◯ **Failed requests**
  Verify invalid requests do not leave partial database changes.
- ◯ **Response format**
  Check that responses use the required message/data structure where applicable.

A checked item should mean that you've actually verified it—not just that you found a corresponding function in the source code.

Step 23: Deploy your API and run the self-check

Once your local API works, the next milestone is deployment. The course autograder cannot access your computer's `localhost`, so your API must be available at a public URL.

Step 23.1: Deploy to Render

Follow the deployment instructions in the course README and DevLab materials.

Your deployment checklist:

1. Push your project to your own GitHub repository.
2. Create a web service on Render connected to that repository.
3. Configure the `MONGODB_URI` environment variable using your MongoDB Atlas connection string.
4. Ensure the service starts using the project's configured start command.
5. Open the deployed URL and confirm the API responds.
6. Test a collection endpoint, such as `/api/tasks`.

Security reminder: Never commit your `.env` file or expose your database password. Configure secrets in Render's environment settings instead.

Step 23.2: Run the course self-check

Open `submit.js` in your project. It contains two URL constants:

- `API_URL`: change this to your own deployed API's base URL.
- `AUTOGRADER_URL`: leave this unchanged.

Then run:

```bash
npm run submit
```

The script prints the test results and a score out of 100. It may take a few minutes, especially if your Render service has been sleeping.

A failed test should include feedback explaining what went wrong. Fix related issues together before rerunning the script, because the autograder has a rate limit.

Important: Running `npm run submit` is a self-check, not the final assignment submission. You must still complete the submission form specified in the README.

Step 24: Prioritize your remaining work

The grading breakdown in your README is a useful guide for deciding what to test most carefully.

Grading area	Weight
Creating and validating users and tasks	20%
`select` on individual resources	7.5%
Updating missing resources	10%
Two-way references and consistency	27.5%
Query parameters	15%
Deleting users and tasks	20%

The biggest area is **two-way reference consistency**, so spend extra time testing task reassignment, completion changes, user updates, and deletions.

One final caution: a perfect self-check score is useful evidence, but it is not a guarantee of your final grade. Make sure your deployment stays available and submit the required form.

What should we do next?

Have you already deployed your API to Render, or are you still working locally? That determines the next practical step.